- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Grow

Python
Logical Operators

Combine conditions and make decisions

Simple
Powerful
Versatile

Satyarthi

1 What are Logical Operators?

Logical operators are used to combine multiple conditions (boolean expressions) in Python. They return a boolean value True or False based on the logic.

2 Why Use Logical Operators?

- Help in decision making
- Used with conditional statements (if, elif, else)
- Useful in loops and complex conditions
- Make the code more efficient and readable
- Widely used in real-world programming

3 Types of Logical Operators

Operator	Name	Description	Example
and	Logical AND	Returns True if both conditions are True	x > 5 and y < 10
or	Logical OR	Returns True if at least one condition is True	x > 5 or y < 10
not	Logical NOT	Returns the opposite of the condition	not(x > 5)

4 Truth Table

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

5 Example Program

Code :

```
1  age = 18
2  has_id = True
3
4  if age >= 18 and has_id:
5      print("You can vote!")
6  else:
7      print("You cannot vote.")
8
9  print(not has_id)
10 print(age < 18 or has_id)
```

Output :

```
You can vote!
False
True
```

6 More Examples

Expression	Result
True and True	True
True and False	False
False and False	False
True or True	True
True or False	True
False or False	False
not True	False
not False	True
(5 > 3) and (2 < 4)	True
(5 < 3) or (2 < 4)	True
not(5 > 3)	False

7 Key Takeaways

- [x] Logical operators return True or False.
- [x] and: True only if both conditions are True.
- [x] or : True if at least one condition is True.
- [x] not : Reverses the result.
- [x] They are widely used in if statements, loops and real-world programs.

8 Did You Know?

Logical operators can be combined:

Example:

```
if age >= 18 and has_id or is_student:
    # More complex condition
```

"Good logic leads to better decisions."

– Satyarthi

Keep Learning
Keep Growing

- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Grow

Python
Logical Operators

Combine conditions and make decisions

Simple
Powerful
Versatile

Satyarthi

1 What are Logical Operators?

Logical operators are used to combine multiple conditions (boolean expressions) in Python. They return a boolean value True or False based on the logic.

2 Why Use Logical Operators?

- Help in decision making
- Used with conditional statements (if, elif, else)
- Useful in loops and complex conditions
- Make the code more efficient and readable
- Widely used in real-world programming

3 Types of Logical Operators

Operator	Name	Description	Example
and	Logical AND	Returns True if both conditions are True	x > 5 and y < 10
or	Logical OR	Returns True if at least one condition is True	x > 5 or y < 10
not	Logical NOT	Returns the opposite of the condition	not(x > 5)

4 Truth Table

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

5 Example Program

Code :

```
1  age = 18
2  has_id = True
3
4  if age >= 18 and has_id:
5      print("You can vote!")
6  else:
7      print("You cannot vote.")
8
9  print(not has_id)
10 print(age < 18 or has_id)
```

Output :

```
You can vote!
False
True
```

6 More Examples

Expression	Result
True and True	True
True and False	False
False and False	False
True or True	True
True or False	True
False or False	False
not True	False
not False	True
(5 > 3) and (2 < 4)	True
(5 < 3) or (2 < 4)	True
not(5 > 3)	False

7 Key Takeaways

- [x] Logical operators return True or False.
- [x] and: True only if both conditions are True.
- [x] or : True if at least one condition is True.
- [x] not : Reverses the result.
- [x] They are widely used in if statements, loops and real-world programs.

8 Did You Know?

Logical operators can be combined:

Example:

```
if age >= 18 and has_id or is_student:
    # More complex condition
```

"Good logic leads to better decisions."

– Satyarthi

Keep Learning
Keep Growing

- [x] Learn
- [x] Practice
- [x] Build Logic
- [x] Improve

Python

Comparison Operators

Compare values and get True or False

Simple
Powerful
Versatile

Satyarthi

1 What are Comparison Operators?

Comparison operators are used to compare two values in Python. They check the relationship between values and return either True or False

2 Why Use Comparison Operators?

- Help in decision making
- Used in conditional statements (if, elif, else)
- Useful in loops and program logic
- Help to compare numbers, strings, variables, and objects
- Essential for real-world problem solving

3 List of Comparison Operators

Operator	Meaning	Example	Result
==	Equal to	5 == 5	True
!=	Not equal to	5 != 3	True
>	Greater than	5 > 3	True
<	Less than	5 < 3	False
>=	Greater than or equal to	5 >= 5	True
<=	Less than or equal to	5 <= 2	False

4 Examples with Different Data Types

Example	Result	
10 == 10	True	
10 != 20	True	
7 > 3	True	
7 < 2	False	
7 >= 7	True	
5 <= 8	True	
'apple' == 'apple'	True	
'apple' != 'mango'	True	
'abc' < 'bcd'	True	(alphabetical order)
'5' == 5	False	(string vs int)

5 Example Program

Code :

```
a = 10
b = 5

print(a == b)          # False
print(a != b)          # True
print(a > b)           # True
print(a < b)           # False
print(a >= b)          # True
print(a <= b)          # False
print('apple' == 'apple') # True
```

Output :

```
False
True
True
False
True
False
True
```

6 Key Takeaways

- [x] Comparison operators return only True or False.
- [x] They can be used with numbers, strings, and variables.
- [x] == checks for equality, != checks for inequality.
- [x] > , < , >= , <= are used to compare values.
- [x] Useful in if conditions, loops, and real-world programs.
- [x] Be careful when comparing different data types (e.g., string and integer).

7 Did You Know?

Strings are compared based on their lexicographical (dictionary) order.

Example: 'apple' < 'mango' → True

(because 'a' comes before 'm')

"Compare today, decide tomorrow."

– Satyarthi

Code
Practice
Repeat

- [x] Learn
- [x] Practice
- [x] Build Projects
- [x] Grow

Python User Input

Simple
Powerful
Versatile

Take input from the user and make your programs interactive

Satyarthi

1 What is User Input in Python?

User input allows the program to take data from the user while running. We use the `input()` function to get input, which is always received as a `string`.

2 Syntax

`input(prompt)`

- prompt (optional) → A message shown to the user.
- The function returns the input as a string.
- We can convert it to other data types using int(), float(), etc.

3 Basic Examples

1 Take a simple input

Code :

```
1  name = input("Enter your name: ")
2  print("Hello, " + name + "!")
```

Output (user enters Satyam)

```
Enter your name: Satyam
Hello, Satyam!
```

2 Take a number and perform operation

Code :

```
1  num = int(input("Enter a number: "))
2  print("Number doubled:", num * 2)
```

Output (user enters 5)

```
Enter a number: 5
Number doubled: 10
```

3 Take multiple inputs in one line

Code :

```
1  a, b = map(int, input("Enter two numbers: ").split())
2  print("Sum:", a + b)
```

Output (user enters 4 6)

```
Enter two numbers: 4 6
Sum: 10
```

4 Converting Input to Different Types

Type	Syntax	Example
String	name = input("Enter name: ")	John
Integer	n = int(input("Enter a number: "))	10
Float	x = float(input("Enter value: "))	3.14
Multiple values	a, b = map(int, input().split())	4 5
List of values	nums = list(map(int, input().split()))	1 2 3

5 Example: Simple Calculator

Code :

```
1  a = float(input("Enter first number: "))
2  b = float(input("Enter second number: "))
3  op = input("Enter operator (+, -, *, /): ")
4  if op == '+':
5      print("Result:", a + b)
6  elif op == '-':
7      print("Result:", a - b)
8  elif op == "*":
9      print("Result:", a * b)
10 elif op == '/':
11     print("Result:", a / b)
12 else:
13     print("Invalid operator!")
```

Sample Run :

```
Enter first number: 10
Enter second number: 5
Enter operator (+, -, *, /): *
Result: 50.0
```

6 Key Takeaways

- [x] Use input() to take user input.
- [x] Input is always a string, so convert it when needed.
- [x] Use int(), float(), etc. for numeric input.
- [x] Use split() to take multiple inputs in one line.
- [x] User input makes your programs interactive and flexible.

7 Did You Know?

- You can customize the input prompt with any message.
- You can take multiple inputs using split().
- User input is widely used in real-world applications like forms, games, and calculators.

"Great programs start with curious users." – Satyarthi

Keep Learning
Keep Growing

Practice
Code
Improve

Python String Methods

Practice Question 4

Small Steps Big Skills!

Question

Topic: **replace()** and **upper()**

What will be the output of the following code?

```
s = "python is fun"
s = s.replace("python", "java")
print(s.upper())
```

A `'PYTHON IS FUN'`

B `'java is fun'`

C `'JAVA IS FUN'`

D `'Java Is Fun'`

"Practice Strings, Master Python!"

Answer: C

Explanation:

First, **replace()** changes **"python"** to **"java"**, so the string becomes:

`"java is fun"`

Next, **upper()** converts all characters to uppercase, so the final output is:

`'JAVA IS FUN'`

Key Concepts Used

- **replace(old, new)** – replaces all occurrences of a substring with another substring.
- **upper()** – converts all characters in a string to uppercase.

Syntax:

```
s.replace("old", "new")
s.upper()
```

Code Today, Better Tomorrow!

By Satyarthi

Learn Practice Improve Grow

Python String Methods

Practice Question 5

Small Steps Big Skills!

Keep Practicing Python!

Question 5

Topic: **count()** method

What will be the output of the following code?

```
s = "banana banana"
print(s.count("a"))
```

A 4

B 5

C 6

D 7

"Strings may look simple, but they can do amazing things with the right methods!"

Answer: C

Explanation:

The **count()** method returns the number of non-overlapping occurrences of a substring in the string.

In the string **"banana banana"**, the letter `'a'` appears 6 times.

```
b a n a n a   b a n a n a
  ↑   ↑   ↑     ↑   ↑   ↑
  1   2   3     4   5   6
```

So, the output is **6.**

About count() Method

- Returns the number of non-overlapping occurrences of a substring.
- It is case-sensitive.
- Syntax: `string.count(substring)`
- It does not modify the original string.

More Examples:

```
s = "hello hello"
print(s.count("l"))  # Output: 4
```

```
s = "Python"
print(s.count("p"))  # Output: 0
```

"Practice Today, Stronger Tomorrow!"

By Satyarthi

Python String Methods

'Small methods, Big possibilities!

String methods are built-in functions that you can use with a string to perform common operations like changing case, searching, replacing, splitting, etc.

Method	Purpose	Example Code	Output
`lower()`	Converts all characters to lowercase	`s = "Hello World"` `print(s.lower())`	`'hello world'`
`upper()`	Converts all characters to uppercase	`s = "Hello World"` `print(s.upper())`	`'HELLO WORLD'`
`strip()`	Removes leading and trailing spaces	`s = "  Python  "` `print(s.strip())`	`'Python'`
`replace()`	Replaces a substring with another	`s = "I love Python"` `print(s.replace("love","like"))`	`'I like Python'`
`split()`	Splits the string into a list	`s = "apple,banana,mango"` `print(s.split(","))`	`['apple', 'banana', 'mango']`
`join()`	Joins elements of a list into a string	`words = ['I', 'love', 'Python']` `print(" ".join(words))`	`'I love Python'`
`find()`	Returns the index of first occurrence (or -1 if not found)	`s = "Python"` `print(s.find("th"))`	`2`
`count()`	Counts the number of occurrences of a substring	`s = "banana"` `print(s.count("a"))`	`3`
`startswith()`	Checks if string starts with given value	`s = "Python"` `print(s.startswith("Py"))`	`True`
`endswith()`	Checks if string ends with given value	`s = "Python"` `print(s.endswith("on"))`	`True`
`title()`	Converts first letter of each word to uppercase	`s = "hello world"` `print(s.title())`	`'Hello World'`
`capitalize()`	Converts first character to uppercase	`s = "python"` `print(s.capitalize())`	`'Python'`
`isalnum()`	Returns True if all characters are letters or digits	`s = "Python123"` `print(s.isalnum())`	`True`
`isalpha()`	Returns True if all characters are letters	`s = "Python"` `print(s.isalpha())`	`True`
`isdigit()`	Returns True if all characters are digits	`s = "12345"` `print(s.isdigit())`	`True`
`isspace()`	Returns True if all characters are spaces	`s = "   "` `print(s.isspace())`	`True`
`swapcase()`	Swaps uppercase to lowercase and lowercase to uppercase	`s = "PyThOn"` `print(s.swapcase())`	`'pYtHoN'`
`center()`	Centers the string with given width and fill character	`s = "Python"` `print(s.center(15, "*"))`	`'***Python***'`
`ljust()`	Left aligns the string	`s = "Python"` `print(s.ljust(10, '-'))`	`'Python----'`
`rjust()`	Right aligns the string	`s = "Python"` `print(s.rjust(10, '-'))`	`'----Python'`

Remember:

- String methods do not change the original string.
- They return a new string.
- Strings are immutable in Python.

"Practice Strings, Build Skills!"

```
s = "Keep Learning"
print(s)
# Happy Coding!
```

By Satyarthi

Python Arrays

Store Multiple Values of the Same Data Type

Learn Practice Build Grow!

What is an Array?

- An **array** is a collection of elements of the **same data type** stored in a sequence.
- Elements are stored in contiguous memory locations.
- Each element is accessed using an **index** (starting from 0).
- In Python, we use the **array module** to work with arrays.
- Arrays are more memory efficient than lists when working with large amounts of data.

Creating an Array

Syntax:

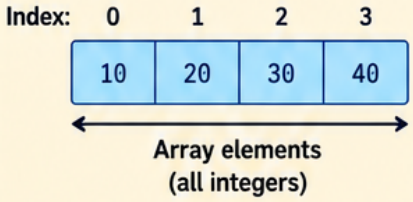
```
from array import array
arr = array('typecode', [elements])
```

Common Typecodes:

Typecode	Data Type	Example
'i'	int (integer)	array('i', [1, 2, 3])
'f'	float (decimal)	array('f', [1.5, 2.5])
'd'	double (float)	array('d', [1.1, 2.2])
'u'	unicode char	array('u', ['a', 'b'])

Example

```
from array import array
# create an integer array
arr = array('i', [10, 20, 30, 40])
print(arr)
# Output:
array('i', [10, 20, 30, 40])
```



Accessing Elements

```
from array import array
arr = array('i', [5, 10, 15, 20])
print(arr[0])    # 5
print(arr[2])    # 15
```

Elements are accessed using index (starting from 0).

Modifying Elements

```
from array import array
arr = array('i', [10, 20, 30])
arr[1] = 100    # change 20 to 100
print(arr)
```

```
# Output:
array('i', [10, 100, 30])
```

Length of an Array

```
from array import array
arr = array('i', [1, 2, 3, 4, 5])
print(len(arr))
```

```
# Output:
5
```

Use `len()` to find the total number of elements.

Practice Question 1

What will be the output of the following code?

```
from array import array
arr = array('i', [10, 20, 30, 40])
print(arr[3])
```

A 10

B 20

C 30

D 40

Practice Question 2

What will be the output of the following code?

```
from array import array
arr = array('i', [1, 2, 3, 4])
arr[0] = 5
print(arr)
```

A array('i', [1, 2, 3, 4])

B array('i', [5, 2, 3, 4])

C array('i', [1, 5, 3, 4])

D array('i', [5, 2, 3])

Key Points

- Array stores elements of the same data type.
- Indexing starts from 0.
- Use the **array** module in Python.
- Arrays are memory efficient.
- Useful for numerical data and large datasets.
- Common typecodes: 'i', 'f', 'd', 'u'.
- Use **len()** to get the size.
- Elements can be accessed and modified using index.

"Organize Data, Solve Bigger Problems!"

Small Steps Big Dreams!

By Satyarthi

Learn Practice Improve Grow

Python Functions

1. Defining and Calling a Function

Small Steps Big Skills!

1. What is a Function?

- A function is a block of code that performs a specific task.
- It helps to make our program organized, reusable and easy to understand.
- We define a function using the `def` keyword and call it by its name.

2. Syntax

Keyword to define a function | Function name (choose any valid name) | Parameters (optional)

```
def function_name(parameters):
    # code to be executed
    return value # optional
```

3. Example: Simple Function

A function that prints a message.

```
def greet():
    print("Hello! Welcome to Python!")
```

Calling the function:

```
greet()
```

Output:

```
Hello! Welcome to Python!
```

Define Once Use Many Times!

4. Example: Function with Parameters

A function that takes a name as a parameter and prints a personalized message.

```
def greet(name):
    print("Hello", name + "!")
```

Calling the function:

```
greet("Aman")
```

Output:

```
Hello Aman!
```

Key Takeaways

- A function is defined using the def keyword.
- You can call a function by its name followed by parentheses ().
- Parameters allow you to pass values to a function.
- Functions make your code reusable and organized.
- The return statement is used to give a value back (optional).

Practice Functions to Write Better Programs!

Code Today, Create a Better Tomorrow!

By Satyarthi

Learn
Practice
Improve
Grow

Python Arrays

Practice Question

Small Steps Big Skills!

Question

What will be the output of the following code?

```
from array import array
arr = array('i', [10, 20, 30, 40, 50])
print(arr[2])
```

A 10

B 20

C 30

D 40

Understand Indexes, Access Data, Build Skills!

Answer: C

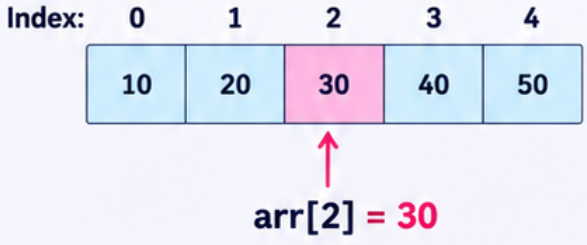
Explanation:

In Python arrays, elements are accessed using an index, starting from 0.

Here, `arr[2]` means the element at index 2, which is **30**.

Visual Representation

```
arr = array('i', [10, 20, 30, 40, 50])
```



Key Points

- Elements are accessed using an index (starting from 0).
- Use `arr[index]` to access a specific element.
- If index is out of range, Python raises an **IndexError**.
- Arrays store elements of the same data type.

Note

- You can also access first and last elements:
  `arr[0]` → 10 (first element)
  `arr[-1]` → 50 (last element)
- Array elements can be accessed, modified and iterated using indexing.

"Practice Today, Stronger Tomorrow!"

By Satyarthi

Learn
Practice
Improve
Grow

Python Arrays

Practice Question

Topic: **Accessing Elements**

Small Steps Big Skills!

Keep Practicing
Keep Growing!

Question

What will be the output of the following code?

```
from array import array
arr = array('i', [5, 15, 25, 35, 45])
print(arr[-1])
```

A 5

B 25

C 35

D 45

"Test your understanding, one question at a time!"

Answer: D

Explanation:

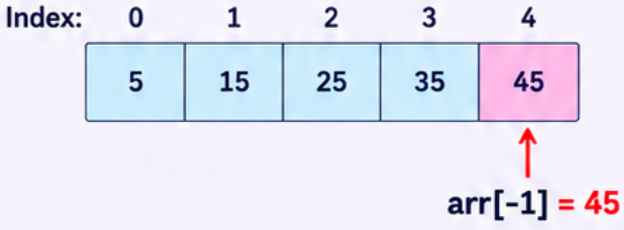
In Python, index -1 refers to the last element of the array.

The array is:

```
arr = [5, 15, 25, 35, 45]
```

So, `arr[-1]` gives the last element, which is **45**.

Visual Representation



Key Points

- Negative indexing starts from the end (-1 is the last element).
- Use `arr[index]` to access elements.
- If index is out of range, Python raises an **IndexError**.
- Arrays store elements of the same data type.

Code Practice Makes Progress!

By Satyarthi

Learn
Practice
Improve
Grow

Python Arrays

Practice Question

Small Steps Big Skills!

Keep Practicing
Keep Growing!

Question

What will be the output of the following code?

```
from array import array
arr = array('i', [10, 20, 30, 40])
arr[1] = 99
print(arr)
```

A `array('i', [10, 20, 30, 40])`

B `array('i', [10, 99, 30, 40])`

C `array('i', [10, 20, 99, 40])`

D `array('i', [99, 20, 30, 40])`

"Arrays let you store, access and modify data efficiently!"

Answer: B

Explanation:

The statement `arr[1] = 99` changes the element at index 1 (second element) from 20 to 99.

So, the array becomes:

```
array('i', [10, 99, 30, 40])
```

Visual Representation

Initially:

Index:	0	1	2	3
	10	20	30	40

After `arr[1] = 99`:

Index:	0	1	2	3
	10	99	30	40

Updated element

Key Points

- Array elements can be modified using index.
- Indexing starts from 0.
- Use the **array** module to work with arrays.
- All elements in an array must be of the same data type.
- Array is more memory efficient than a list.

Syntax Reminder

```
from array import array
arr = array('i', [1, 2, 3])
arr[index] = value   # modify element
print(arr)
```

Practice More, Code Better!

Consistent Practice Builds Strong Programmers!

By Satyarthi